Optimisation des sites internet

Performance, UX et SEO

ITUniversity | Cours de Développement Web

Plan du cours

- 1. Introduction — Pourquoi optimiser ?
- 2. Indicateurs clés (KPIs)
- 3. Outils d'analyse et diagnostic
- 4. Optimisation Frontend
- 5. Optimisation du chargement
- 6. Optimisation réseau
- 7. Optimisation serveur
- 8. Architecture web moderne
- 9. Optimisation mobile
- 10. UX et perception de performance
- 11. SEO et accessibilité
- 12. Sécurité et performance
- 13. Tests et suivi
- 14. Cas pratique (TP)
- 15. Bonnes pratiques avancées
- 16. Conclusion

1a Introduction — Définition & enjeux

- **Définition de la performance web**

Temps de réponse, fluidité, disponibilité des ressources

Perçue par l'utilisateur dès la première seconde

- **Enjeux : Expérience utilisateur (UX)**

Satisfaction, rétention, engagement

- **Enjeux : Référencement naturel (SEO)**

La performance est un facteur de classement Google

1b Introduction — Contexte & business

- **Enjeux : Taux de conversion (business)**

+1s de chargement = -7% de conversions

Impact direct sur le chiffre d'affaires

- **Contexte mobile et pays émergents**

Réseaux lents (3G/2G), appareils moins puissants

Plus de 60% du trafic mondial est mobile

Approche mobile-first impérative

2a Les indicateurs clés de performance — Métriques de base

- **Temps de chargement total**

Durée complète jusqu'au rendu de la page

- **TTFB (Time To First Byte)**

Temps entre la requête et le premier octet reçu

Indicateur de la réactivité serveur

- **FCP (First Contentful Paint)**

Premier élément visuel affiché

2b Les indicateurs clés de performance — Core Web Vitals

- **LCP (Largest Contentful Paint)**

Chargement du contenu principal — cible < 2.5s

- **INP / FID (Interaction to Next Paint)**

Réactivité aux interactions — cible < 200ms

- **CLS (Cumulative Layout Shift)**

Stabilité visuelle — cible < 0.1

- **Interprétation des scores**

Vert (Good) / Orange (Needs Improvement) / Rouge (Poor)

3a Outils d'analyse — DevTools navigateur

- **Onglet Réseau**

Waterfall de chargement des ressources

Filtrage par type (JS, CSS, images, XHR…)

- **Console**

Erreurs JS, avertissements, logs

- **Onglet Application**

Cache, stockage local, Service Workers, cookies

3b Outils d'analyse — Outils externes

- **Google PageSpeed Insights**

Score 0-100 + recommandations priorisées

Données terrain (CrUX) + données lab

- **Lighthouse**

Audit complet : performance, accessibilité, SEO, PWA

- **GTmetrix**

Analyse détaillée + historique + tests depuis plusieurs localisations

- **Lecture des résultats**

Cascade de chargement, opportunités d'amélioration

4a Optimisation Frontend — Ressources statiques

- **Minification (HTML, CSS, JS)**

Suppression des espaces, commentaires, noms raccourcis

- **Compression des images**

WebP : -30% vs JPEG, -70% vs PNG

AVIF : encore plus compressé, support croissant

- **Redimensionnement adapté**

Ne pas charger 2000px pour afficher 400px

- **Sprites CSS**

Regrouper les icônes en une seule image = moins de requêtes

4b Optimisation Frontend — CSS & JavaScript

- **CSS : Critical Path**

Styles above-the-fold inlinés dans le <head>

Reste chargé en différé (preload + onload)

- **JS : defer et async**

defer — exécution après parsing HTML complet

async — exécution parallèle, sans ordre garanti

- **Code splitting & Tree shaking**

Charger uniquement le code nécessaire à la page

Bundlers : Webpack, Vite, Rollup

5a Optimisation du chargement — Lazy Loading

- **Images : attribut loading='lazy'**

Chargement déclenché à l'approche du viewport

- **Vidéos : chargement à la visibilité**

Éviter de précharger des vidéos non vues

- **Composants : import dynamique**

React, Vue : import() à la demande

- **Intersection Observer API**

Déclencher le chargement au moment exact de la visibilité

5b Optimisation du chargement — Rendu progressif

- **Prioriser le contenu above the fold**

Rendre visible le haut de page en premier

Différer tout ce qui est hors écran

- **Skeleton loading**

Placeholder animé pendant le chargement

Améliore fortement la perception de rapidité

- **Preload / Prefetch**

<link rel='preload'> pour les ressources critiques

<link rel='prefetch'> pour les pages probables suivantes

6a Optimisation réseau — Requêtes & CDN

- **Réduction des requêtes HTTP**

Regroupement CSS/JS, sprites, data URIs

- **Regroupement des appels API (batching)**

Combiner plusieurs requêtes en une seule

- **CDN (Content Delivery Network)**

Distribution géographique des ressources statiques

Serveur le plus proche = latence réduite

6b Optimisation réseau — Compression & protocoles

- **Compression des transferts**

Gzip — compression universelle

Brotli — plus efficace, supporté par tous les navigateurs modernes

- **HTTP/2**

Multiplexage, server push, compression headers

- **HTTP/3**

Basé sur QUIC, réduction de latence sur mauvais réseaux

- **Keep-Alive**

Réutilisation des connexions TCP existantes

7a Optimisation serveur — Hébergement & cache

- **Choix de l'hébergement adapté**

Mutualisé / VPS / Cloud (AWS, GCP, Azure…)

Hébergement proche des utilisateurs cibles

- **Réduction du TTFB**

Traitement côté serveur optimisé

Limiter les opérations bloquantes

- **Mise en cache côté serveur**

Full page cache, fragment cache

Cache HTTP (Cache-Control, ETag)

7b Optimisation serveur — Proxy & base de données

- **Reverse proxy — Nginx**

Serveur de fichiers statiques ultra-performant

Compression, cache, load balancing

Terminaison SSL/TLS

- **Base de données optimisée**

Index sur les colonnes fréquemment requêtées

Requêtes préparées, éviter les N+1

Pool de connexions (PgBouncer, HikariCP…)

8a Architecture web moderne — SPA / MPA / SSR

- **SPA (Single Page Application)**

Navigation fluide, JS côté client

Problèmes SEO et TTI initial plus lent

- **MPA (Multi-Page Application)**

Plusieurs pages HTML, rendu serveur classique

Meilleur SEO natif, moins de JS côté client

- **SSR — Server Side Rendering**

HTML généré côté serveur à chaque requête

Meilleur SEO et 1er affichage rapide

8b Architecture web moderne — SSG & Frameworks

- **SSG — Static Site Generation**

Pages générées à la compilation (build time)

Performances maximales, hébergement simple

Idéal pour les contenus peu changeants

- **Next.js (React)**

SSR + SSG hybride, ISR (revalidation incrémentale)

- **Nuxt.js (Vue)**

Idem pour l'écosystème Vue.js

- **Astro, SvelteKit**

Alternatives modernes orientées performance

9a Optimisation mobile — Stratégie & images

- **Approche Mobile-first**

Concevoir d'abord pour les petits écrans

Puis enrichir pour les écrans larges

- **Images responsives**

Attribut srcset — plusieurs résolutions proposées

Attribut sizes — taille selon le viewport

Balise <picture> pour les formats alternatifs

- **Adaptation aux réseaux lents**

Limiter les ressources lourdes en 3G

9b Optimisation mobile — Scripts & tests

- **Réduction des scripts lourds**

Éviter les librairies inutiles (lodash, moment…)

Préférer les alternatives légères ou le vanilla JS

- **Service Workers + cache offline**

Permettre l'accès hors connexion

PWA (Progressive Web App)

- **Tests sur appareils réels**

Ne pas se fier uniquement à l'émulateur DevTools

Tester sur appareils bas de gamme réels

10a UX et perception — Temps réel vs perçu

- **Temps réel vs temps perçu**

Un site peut sembler plus rapide qu'il n'est

La perception est aussi importante que la mesure

- **Feedback utilisateur immédiat**

Spinners, progress bars, messages de chargement

- **Optimistic UI**

Affichage anticipé avant confirmation serveur

Ex : like instantané sur réseau social

10b UX et perception — Animations & rendu progressif

- **Animations et transitions**

Donner l'impression de fluidité

Utiliser CSS (GPU) plutôt que JS (CPU)

Éviter les animations bloquant le thread principal

- **Rendu progressif**

Afficher le contenu dès que possible

Above the fold en priorité

- **Skeleton loading**

Placeholder structuré pendant le chargement

Réduit l'anxiété de l'utilisateur

11 SEO et accessibilité

- **SEO — Référencement naturel**

Balises <title> et <meta description> uniques

Structure Hn correcte (H1 unique, hiérarchie logique)

Performance = facteur de classement Google

Sitemap XML, robots.txt

- **Accessibilité (a11y)**

Contraste des couleurs (ratio WCAG minimum 4.5:1)

Navigation clavier — tous les éléments interactifs accessibles

Attributs alt sur les images

Normes WCAG 2.1 (Niveaux A, AA, AAA)

12a Sécurité et performance — HTTPS & SEO

- **HTTPS / TLS**

Chiffrement des échanges client–serveur

Obligatoire pour HTTP/2

Certificats Let's Encrypt (gratuits, automatisés)

- **Impact sur le SEO**

Google favorise les sites en HTTPS depuis 2014

Mixed content bloque les ressources HTTP sur HTTPS

12b Sécurité et performance — Sécurisation des ressources

- **Content Security Policy (CSP)**

Restreindre les sources de scripts, styles, images

- **Subresource Integrity (SRI)**

Vérifier l'intégrité des ressources CDN

- **Headers de sécurité HTTP**

HSTS — forcer HTTPS

X-Frame-Options — prévenir le clickjacking

X-Content-Type-Options — prévenir le MIME sniffing

13a Tests et suivi — Monitoring & logs

- **Suivi des performances dans le temps**

Éviter les régressions après mises à jour

- **Analyse des logs serveur**

Identifier requêtes lentes, erreurs 5xx/4xx

- **Monitoring continu**

Alertes automatiques (Uptime Robot, Pingdom…)

Tableaux de bord Grafana + Prometheus

13b Tests et suivi — Benchmarks & tests de charge

- **Tests avant / après optimisation**

Benchmark initial, application des correctifs

Comparaison des scores Core Web Vitals

- **Tests de charge (stress testing)**

Apache JMeter — scénarios complexes

k6 — scripts JS, CI-friendly

Locust — tests en Python

- **Intégration CI/CD**

Lighthouse CI — bloquer si score < seuil

14 Cas pratique — Travaux Pratiques (TP)

1	**Analyse d'un site existant** Lancer PageSpeed Insights / Lighthouse — noter les scores initiaux
2	**Identification des problèmes** Lister ressources lourdes, requêtes bloquantes, images non optimisées
3	**Application des optimisations** Minification, compression images, lazy loading, cache
4	**Comparaison des résultats** Mesures Avant / Après — scores Core Web Vitals, temps de chargement

15a Bonnes pratiques avancées — Budget & PWA

- **Performance budget**

Définir un budget de poids/temps à ne pas dépasser

Ex : JS < 200KB, LCP < 2.5s

Intégration dans le pipeline CI/CD

- **Service Workers et PWA**

Cache offline — fonctionnement sans connexion

Notifications push, installation sur écran d'accueil

Progressive Web App — installable comme appli native

15b Bonnes pratiques avancées — Cache serveur & BDD

- **Cache avancé côté serveur**

Redis — cache clé/valeur en mémoire

Varnish — reverse proxy cache HTTP full-page

Stratégies : TTL, invalidation, cache-aside

- **Optimisation base de données**

Requêtes préparées, index composites

Connexions poolées (réduction overhead)

EXPLAIN ANALYZE — identifier les requêtes lentes

Pagination, cursors plutôt que OFFSET sur grands datasets

16

Conclusion

- Optimiser = technique + UX + SEO — une approche globale
- Performance continue : mesurer, corriger, itérer
- Mobile-first et accessibilité : impératifs modernes